

Python Code

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import re

# Helper functions
def clean_text(text):
    """Cleans the given text by removing unwanted symbols.

    Args:
        text (str): The input text string.

    Returns:
        str or np.nan: The cleaned text string, or np.nan if the input text is None.
    """
    if text:
        # Remove leading and trailing spaces, parentheses, and newlines.
        return text.strip(' ()').replace('\n', '').strip()
    else:
        # Return np.nan for None input
        return np.nan

def extract_year(text):
    """Extracts the year from a given text string.

    Args:
        text (str): The input text string.

    Returns:
        int or nan: The extracted year as an integer, or NaN if no year is found.
    """
```

```

# Regular expression patterns to match different year formats
year_patterns = [
    r"\b\d{4}\b", # Four digits alone (e.g., 2023)
    r"\b(?:19|20)\d{2}\b", # Four digits starting with 19 or 20 (e.g., 1990, 2021)
    r"\b(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)\s+\d{4}\b", # Month and year (e.g., Jan 2024)
    r"\b\d{4}\s+(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)\b", # Year and month (e.g., 2023 Jan)
    r"\b(?:\d{1,2}\s+)?(?:Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)\s+\d{4}\b", # Day, month, and year
    (e.g., 25 Dec 2023)
]

for pattern in year_patterns:
    match = re.search(pattern, text)
    if match:
        year_str = match.group()
        try:
            year = int(year_str)
            if 1900 <= year <= 2100: # Ensure valid year range
                return year
        except ValueError:
            pass # Ignore invalid year formats

    return np.nan # No year found

# Q1 - On craigslist search for "Toyota Camry" and select search 100 miles from your location. (Using BU campus
# ZIP code)

url =
"https://boston.craigslist.org/search/cta?postal=02215&query=toyota%20camry&search_distance=100#search=1~gall
ery~0~0"

# Q2 - Get the URL and utilize BeautifulSoup to build a data frame with the parsed data from your search.

def beautiful_soupify(url):
    """Fetches the HTML content of a given URL and parses it using BeautifulSoup.

    Args:
        url (str): The URL of the web page to fetch.

```

Returns:

BeautifulSoup object or None: The parsed BeautifulSoup object if the request is successful, or None if the request fails.

"""

```
# Send a GET request to the specified URL
```

```
response = requests.get(url)
```

```
# Check if the request was successful (status code 200)
```

```
if response.status_code == 200:
```

```
    # Parse the HTML content using BeautifulSoup
```

```
    soup = BeautifulSoup(response.content, 'html.parser')
```

```
    return soup
```

```
else:
```

```
    print("Connection request failed. Status code:", response.status_code)
```

```
    return None # Return None to indicate a failed request
```

Q3 - Your data frame should look like this example, with Description, Location, Price, and Year (when each data point is not available, use "null" numpy.nan).

```
def load_results(soup):
```

```
    """Extracts Craigslist listing information from a BeautifulSoup object.
```

Args:

soup (BeautifulSoup object): The parsed BeautifulSoup object containing the listing data.

Returns:

list: A list of dictionaries, each representing a listing with information like description, location, price, and year.

"""

```
listings = [] # Initialize an empty list to store the extracted listings
```

```
# Find all listing elements on the page
```

```
for result in soup.find_all('li', class_='cl-static-search-result'):
```

```
    # Extract the description, location, and price information from each listing
```

```
    description_tag = result.find('div', class_='title')
```

```

description = clean_text(description_tag.text) if description_tag else np.nan

location_tag = result.find('div', class_='location')
location = clean_text(location_tag.text) if location_tag else np.nan

price_tag = result.find('div', class_='price')
price = clean_text(price_tag.text) if price_tag else np.nan

# Convert the price to an integer, removing '$' and ','
if price != np.nan:
    price = price.replace('$', '').replace(',', '')
    try:
        price = int(price)
    except ValueError:
        price = np.nan # If conversion fails, set to NaN

# Extract the year from the description
year = extract_year(description)

# Add the listing information to the list
listings.append({
    'Description': description,
    'Location': location,
    'Price': price,
    'Year': year
})

return listings

def create_df(listings):
    """Creates a pandas DataFrame from a list of listing dictionaries.

    Args:
        listings (list): A list of dictionaries, each representing a listing with information like description, location, price, and year.

    Returns:

```

```

pandas.DataFrame: A DataFrame containing the listing data, with appropriate data types for price and year.
"""

# Create a DataFrame from the list of listings
df = pd.DataFrame(listings, columns=['Description', 'Location', 'Price', 'Year'])

# Convert the 'Price' and 'Year' columns to numeric, handling potential conversion errors
df['Price'] = pd.to_numeric(df['Price'], errors='coerce').astype('Int64')
df['Year'] = pd.to_numeric(df['Year'], errors='coerce').astype('Int64')

return df

soup = beautiful_soupify(url)

if soup:
    listings = load_results(soup)

    # Build DataFrame
    df = create_df(listings)
    print(df.head()) # Print first 5 results

# Q4 - Replace the missing values in "Year" column with the median value of the "Year" column.

df.fillna({'Year': df['Year'].median()}, inplace=True)

# Q5 - Plot the scatter plot for "Year" vs "Price".

# Create the figure with desired size
fig, ax = plt.subplots(figsize=(16, 9))

# Scatter plot with transparency
ax.scatter(df['Year'], df['Price'], alpha=0.5)

# Set title, labels, and display the plot
ax.set_title('Craigslist Toyota Camry Price vs Model Year')
ax.set_xlabel('Year')

```

```

ax.set_ylabel("Price (in USD)")

# Save the figure as a PNG image
plt.savefig('craigslist_toyota_camry_price.png')

# Q6 - Get the Pearson Correlation between "Year" and "Price".

pearson_corr = df['Year'].corr(df['Price'])
print(f'Pearson Correlation between "Year" and "Price": {pearson_corr}')

```

Terminal Output

```

jirissman@crc-dot1x-nat-10-239-227-176 ~ % /usr/local/bin/python3 /Users/jirissman/Documents/coursework/BU/CS688/Rissman_Hw2.py
0          2004 TOYOTA CAMRY INSPECTED RUNS GREAT  NEW HAMPSHIRE  3000  2004
1          2002 Toyota Camry                      Belmont      1500  2002
2  TOYOTA CAMRY INSPECTION READY AUTOMATIC RUNS L...  NEW HAMPSHIRE  2500  <NA>
3          2018 Toyota Camry LE 4dr Sedan          NaN      15995  2018
4          2009 Toyota Camry great gas mileage!    Raymond    6200  2009
Pearson Correlation between "Year" and "Price": 0.5831986543186389
jirissman@crc-dot1x-nat-10-239-227-176 ~ %

```

Scatter Plot

